

I. Architecture and Communication

The core requirement is to develop an intelligent agent orchestrated using **LangGraph** and served by a Python web application.

- **Application Framework (Choice):** The agent service must be hosted within a standard **Django Ninja** (preferred) **OR** a **Django** application (using Django REST Framework). The candidate must clearly state the chosen framework.
- **Agent Orchestration:** The agent's logic and state management must be handled by **LangGraph**.
- **Database Flexibility:** The relational database can be either **PostgreSQL** or **SQLite**. The candidate must clearly document the chosen option.
- **LLM Choice and Follow-up:**
 - The developer has **full flexibility** to choose **any LLM model or architecture** to power the agent.
 - The developer should expect a **follow-up interview** to dive deeper into the technical decisions made regarding the **LangGraph design** and the **specific LLM model/architecture** selected.

II. API Design Standards

The Agent API must strictly follow **RESTful best practices** and align with industry standards for security and performance.

- **Resource Naming:** Use **nouns** (plural) for resource collections.
- **HTTP Methods:** Correctly map **HTTP methods** to actions. A single **POST** endpoint is expected for submitting a new user query to the agent.
- **Status Codes:** Use appropriate **HTTP Status Codes** for responses (e.g., 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error).
- **Authentication (Mandatory):** All communication with API endpoints must be secured using **JSON Web Token (JWT)** authentication, returning a **401 Unauthorized** response if the token is missing or invalid.

III. Agent Capabilities

The LangGraph agent must function as a router, deciding whether to execute a Text-to-SQL task or a Document RAG task based on the user's query.

A. Text-to-SQL (T2SQL) Requirement

The agent must interpret natural language queries into SQL using Vanna.

- **T2SQL Library:** The **Vanna** framework must be used to provide the Text-to-SQL capability.
- **Vector Store: ChromaDB** must be used as the vector store to host Vanna's training data (DDL, documentation, SQL examples).
- **Agent Capability:** The agent must interpret a natural language query, utilize a **Vanna tool** to generate and execute the appropriate SQL query against the relational database, and return a meaningful, natural language response based on the result.

- **Data Handling:** The chosen relational database should contain a sample schema and data sufficient to demonstrate the T2SQL capability.

B. Document RAG Requirement (Brochures)

The agent must be able to retrieve information directly from provided unstructured documents (brochures).

- **Vector Store:** The processed brochure data must be stored and searched using **ChromaDB**.
- **Agent Capability:** The LangGraph agent must utilize a **Document RAG tool** to perform a semantic search against the brochure vectors in ChromaDB, retrieve the most relevant text chunks, and use the LLM to synthesize an accurate, natural language answer.

IV. Development Requirements

- **Codebase:** A complete, working codebase containing the application framework and the full LangGraph agent.
- **Dependencies:** A clear and reproducible list of project dependencies (e.g., `requirements.txt`).
- **Document Ingestion API (Mandatory):** A dedicated **upload endpoint** must be created to handle the ingestion of brochure documents. This endpoint will trigger the following pipeline:
 1. **File Upload:** Accepts the brochure file(s).
 2. **Chunking/Splitting:** Implements a strategy (e.g., recursive text splitting) to break the brochures into manageable chunks.
 3. **Embedding:** Uses an appropriate **Embedding Model** (must be documented) to convert the text chunks into vectors.
 4. **Storage:** Stores these vectors, the original chunks, and relevant metadata in **ChromaDB**.
- **Documentation:** Clear instructions on environment setup, configuration of both databases, and usage of the **Document Ingestion API**.

V. Testing and Evaluation

A robust testing and evaluation suite is mandatory.

- **Testing Framework (Mandatory):** All tests must be written using **Pytest**.
- **Unit/Integration Testing:** Thorough tests for the API workflow, Document Upload/Ingestion flow, T2SQL flow, and the Document RAG retrieval and generation.
- **Agent Evaluation (DeepEval):**
 - The agent's performance must be evaluated using the **DeepEval** framework.
- **Evaluation Results:** The **DeepEval agent score results** must be captured and shared in a separate, dedicated file (e.g., `agent_evaluation_scores.json`).
- **Reproducibility:** The evaluation process must be **easily reproducible** via a single script or command (e.g., `pytest run_eval.py`).

VI. Deployment and Submission

The final deliverable consists of two parts.

- **Hosting Platform:** The entire application must be deployed to some cloud environment that is public facing. You can use **Render** (<https://render.com/>) as a free alternative.
- **Deployment Status:** The deployed application must be live and fully functional upon submission.
- **Submission Artifacts (Mandatory):**
 1. The **live link** to the hosted application.
 2. A **ZIP file** containing the complete source code for the challenge.

VII. Bonus Criteria

Candidates will receive **bonus points** for excellence in the following areas:

Criterion	Description
Clean Architecture	Clear separation of concerns (e.g., distinct layers for API views, LangGraph logic, and data ingestion/tools). Proper use of interfaces and abstractions.
OOP Design	The project must be structured using classes and follow OOP best practices to enhance modularity and maintainability.
Documentation	Comprehensive, clear, and easy-to-follow documentation, particularly for setup, running the agent, and API usage (e.g., using Swagger/OpenAPI, or detailed README).
Organization & Conciseness	Logical file structure, idiomatic Python code, effective use of environment variables, and minimizing unnecessary complexity.
Error Handling	Robust error handling within the LangGraph nodes and API, especially for database connection failures or LLM tool-calling issues.

Submission email:

Submit your final solution with necessary docs and links to careers@proplens.ai

Please write to the same email ID for any clarifications you might have.